CS3081 Cloud Computing: Mini Project Report

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Student Name: Morri Peddaiah

Roll Number: 230101051

Course: Cloud Computing (CS3081)

Academic Year: 2026-2027

Executive Summary

This report documents the design, implementation, and empirical evaluation of an end-to-end cloud-native machine learning pipeline. The project addresses the challenge of running natural language understanding (NLU) workloads on resource-constrained edge/cloud infrastructure. Specifically, sequence-level knowledge distillation (KD) was employed to compress a 3.8-billion-parameter teacher language model (`Phi-3.5-mini-instruct`) into an ultra-compact 70-million-parameter causal language model (`EleutherAI/pythia-70m`). The student model was fine-tuned to extract structured intents and entities from English voice assistant commands under strict schema constraints (exactly 4 keys: `intent`, `date`, `time`, `place`).

The complete software lifecycle—from parallel teacher labelling and automated data sanitization to containerization, Kubernetes orchestration on a local k3d cluster, stateless network access control, CI/CD automated quality gates, and load testing—was implemented and validated.

Task 1: Object Storage & Parallel Teacher Labelling

1.1 Object Storage Architecture with SeaweedFS

For local S3-compatible object storage, SeaweedFS was chosen and deployed within the dedicated `data` namespace. SeaweedFS provides a lightweight, highly responsive storage layer with native support for AWS S3 v4 API signatures.

Two primary buckets were initialized:

1. `datasets`: Stores raw command pools (`commands_pool.jsonl`), worker shard outputs (`shards/shard_*.jsonl`), and versioned cleaned datasets (`v1/train.jsonl`).
2. `models`: Hosts trained model checkpoints, tokenizer configurations, and SHA-256 verification manifests (`v1/model.sha256`).

1.2 Sharding Mechanism and Parallel Worker Execution

The raw command pool consists of 3,000 voice assistant commands drawn from the SLURP benchmark. To expedite teacher labelling while honoring zero-temperature deterministic inference, a multi-worker distributed architecture was configured:

- Each worker pod is assigned two environment variables: `SHARD_INDEX` and `SHARD_COUNT`.
- Partitioning is performed deterministically using modulo index assignment:

$$\text{assigned}(\text{cmd}_i) \iff i \pmod{\text{SHARD_COUNT}} == \text{SHARD_INDEX}$$

- Two workers (`SHARD_COUNT=2`, indices 0 and 1) were executed concurrently within the `pipeline` namespace, querying an Ollama server hosting `Phi-3.5-mini-instruct` with fixed inference parameters:
- Temperature: 0.0
- Random Seed: 42
- Worker prompt design (`prompts/teacher_prompt.txt`) explicitly enforces the 4-key JSON output format and slot extraction rules. Shard results were uploaded directly to `s3://datasets/shards/shard_0.jsonl` and `s3://datasets/shards/shard_1.jsonl`.

1.3 Strict Data Cleaning Pipeline (`worker/clean.py`)

Because large language models can occasionally produce malformed syntax or hallucinate ungrounded entity tokens, the output shards were processed through a multi-stage validation filter:

1. **JSON Syntax Validity**: The response must parse into a valid JSON dictionary.
2. **Key Set Invariance**: The dictionary must contain exactly `{"intent", "date", "time", "place"}`. No missing or superfluous keys.
3. **Intent Domain Verification**: The `intent` value must belong to the predefined set of 8 supported intents:
- `alarm_set`, `alarm_query`, `calendar_set`, `calendar_query`, `calendar_remove`, `weather_query`, `datetime_query`, `recommendation_events`.
4. **Anti-Hallucination Grounding**: Every non-null slot string (`date`, `time`, `place`) must appear verbatim (case-insensitive substring) in the original command text.
5. **Exact Deduplication**: Identical normalized user command texts appearing across different IDs were deduplicated to prevent overfitting.

Cleaning Results Summary:

- Total records read: 3,000
- Invalid JSON / parsing failures: 42
- Key set mismatches: 28
- Disallowed / hallucinated intents: 19
- Hallucinated slot strings: 61

- Duplicate command texts: 314
- Final valid curated training records: 2,536 (First 1,500 selected for distillation)
- Output artifact: `datasets/v1/train.jsonl`
- SHA-256 checksum: Verified and recorded in `datasets/v1/train.sha256`.

Task 2: Training the Student Model

2.1 Distillation Paradigms: Sequence-Level vs. Logit-Based Distillation

A central conceptual question in knowledge distillation is the choice between **Sequence-Level Knowledge Distillation** (Kim & Rush, 2016) and **Logit-Based (Token-Level) Distillation** (Hinton et al., 2015; Chen et al., 2020):

Dimension	Sequence-Level KD (Used Here)	Logit-Based (Token-Level) KD
Training Objective	Standard cross-entropy loss against teacher generated outputs $\mathcal{L}_{seq} = -\sum_{t} \log P_S(y_t^* \mid y_{<t}^*, x)$ where $y^* = \arg\max P_T(y \mid x)$	Combined loss: $\alpha \mathcal{L}_{CE} + (1-\alpha) T^2 \mathcal{L}_{KL}(P_S^{(T)} \parallel P_T^{(T)})$ matching soft probability distributions
Compute Overhead	**Low**. Teacher runs offline once to generate targets. Student trains as a standard causal LM without the teacher in memory.	**Very High**. Teacher and student must run forward passes simultaneously in GPU/CPU memory to compute KL divergence over the entire vocabulary.
Vocabulary Flexibility	Completely model-agnostic. Teacher and student can have different tokenizers and vocabulary sizes (e.g., Phi-3.5 vs Pythia).	Requires identical or projected token vocabularies.
Resource Suitability	Ideal for CPU-only training and local cloud setups with commodity hardware.	Requires multi-GPU clusters to hold multi-billion parameter teacher and student simultaneously.

Because our target deployment environment is CPU-only, **Sequence-Level KD** was selected. It allows offline teacher label generation and fast, efficient student fine-tuning.

2.2 Hyperparameters and Training Configuration

Student training utilized `EleutherAI/pythia-70m` initialized from Hugging Face:

- Base Architecture: 70M parameters, 6 layers, 8 attention heads, hidden dimension 512.
- Input Formatting:

```
Command: <command_text>
JSON: {"intent": "...", "date": ..., "time": ..., "place": ...}
```

- Hyperparameters (`training/config.yaml`):
- Optimizer: AdamW ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$)
- Weight Decay: 0.01
- Learning Rate: 5×10^{-4} with linear warmup (5% of steps) and linear decay
- Batch Size: 16 (Gradient accumulation steps: 2, effective batch size: 32)
- Max Sequence Length: 128 tokens
- Epochs: 3
- Random Seed: 42
- Device: CPU (Intel Core / AMD x86_64)

2.3 Empirical Evaluation on `data/validation.jsonl`

Evaluation was executed across 100 gold-labelled validation samples using `tools/score.py`:

Metric (%)	Untrained Pythia-70M	Phi-3.5 Teacher	Distilled Student (v1)	Distilled Student (v2)
JSON Validity	12.00	99.00	**97.00**	**98.00**
Intent Accuracy	14.00	92.00	**86.00**	**88.00**
Date Accuracy	10.00	91.00	**84.00**	**86.00**
Time Accuracy	8.00	90.00	**82.00**	**85.00**
Place Accuracy	11.00	93.00	**85.00**	**87.00**
Full Frame Accuracy	2.00	85.00	**76.00**	**79.00**

Analysis: The untrained Pythia-70M model fails almost completely on JSON syntax and intent recognition. Through sequence-level KD on 1,500 samples, JSON validity rises dramatically from 12% to 97%+, comfortably exceeding the 95% quality gate requirement. Full frame accuracy reaches 76–79%, demonstrating that a 70M parameter model can effectively memorize and generalize structured extraction patterns.

Task 3: Inference Service & Containerization

3.1 Service Implementation (`service/app.py`)

The inference engine is constructed with FastAPI and Uvicorn:

- **POST `/parse`**: Accepts JSON payload `{"text": "<command>"}`. Prompts the loaded Pythia-70M model, extracts generated tokens, validates the resulting object against the 4-key schema, and returns the parsed result.
- Required Response Headers:
- `X-Model-Version`: Reflects active model version (`v1` or `v2`).
- `X-Served-By`: Returns the container hostname / Kubernetes pod name.
- Schema Enforcement: If the model output fails schema validation, HTTP 500 is immediately returned.
- Fault Injection: When `FAULT_INJECT=1`, the service intentionally injects invalid output `"not json"`, triggering a 500 status code for chaos testing.
- **GET `/healthz`**: Liveness probe returning HTTP 200 `{"status": "ok"}` as long as the web process responds.
- **GET `/readyz`**: Readiness probe returning HTTP 503 while the model weights are downloading or initializing, and HTTP 200 `{"status": "ready"}` only after the model is resident in CPU memory.

3.2 Dynamic Weight Loading from Object Storage

Rather than baking multi-hundred-megabyte model weights into the container image, the service downloads weights dynamically upon startup from SeaweedFS S3 (`s3://models/v1/`) using credentials stored in a Kubernetes Secret. This adheres to 12-Factor App design principles and decouples code from configuration/state.

3.3 Container Optimization (<2 GB Image)

The `Dockerfile` employs multi-stage compilation:

- Base: `python:3.11-slim`
- PyTorch: CPU-only wheels (`--extra-index-url https://download.pytorch.org/whl/cpu`), avoiding 3+ GB of unnecessary CUDA/cuDNN binaries.
- Final Image Size: **~1.15 GB** (well below the 2.0 GB project ceiling).
- Security: Non-root execution under UID 1000 (`appuser`).

Task 4: Kubernetes Deployment & Local Cloud Orchestration

4.1 Topology and Manifests (`k8s/`)

The deployment architecture is organized across three namespaces:

- `data`: SeaweedFS master, volume, and filer services.
- `pipeline`: Parallel worker batch jobs and data cleaning jobs.
- `app`: Production inference service, Traefik ingress, and HPA.

Key Kubernetes Objects:

1. `Deployment`: Configured with 2 replicas, rolling update strategy (`maxSurge: 1`, `maxUnavailable: 0`), CPU request `500m`, CPU limit `1000m` (1 vCPU), memory request `512Mi`, memory limit `1Gi`.
2. `Liveness Probe`: `HTTP GET /healthz` (initial delay: 10s, period: 10s).
3. `Readiness Probe`: `HTTP GET /readyz` (initial delay: 15s, period: 5s).
4. `Service`: ClusterIP on port 8000.
5. `Ingress`: Routes external port 8080 traffic via Traefik to `command-parser-service:8000`.
6. `HPA`: Configured with `minReplicas: 1`, `maxReplicas: 2`, targeting 70% CPU utilization.

4.2 Cold Start Analysis

Cold start measurements were recorded across 3 independent pod launch cycles:

- Run 1: 4.82 seconds
- Run 2: 4.15 seconds
- Run 3: 4.39 seconds
- **Median Cold Start Time: 4.39 seconds**

The readiness probe accurately shielded traffic until the model was completely loaded in memory.

4.3 Rolling Updates and Zero-Downtime Rollback

A rolling update from `v1` to `v2` was initiated using `kubectl set image deployment/command-parser parser=command-parser:v2`.

- During update: Because `maxUnavailable=0`, the existing `v1` pods remained active and served incoming requests until new `v2` pods passed their `/readyz` probes.
- Locust monitoring confirmed zero dropped connections (0.0% error rate).
- Rollback was demonstrated via `kubectl rollout undo deployment/command-parser`, seamlessly restoring `v1`.

4.4 Fault Recovery and Load Balancing

• **Pod Deletion Recovery**: Deleting an active pod (`kubectl delete pod command-parser-...`) triggered immediate replica reconciliation by the Deployment controller. A replacement pod reached ready state within 5.2 seconds with no service interruption.

• **Load Balancing Verification**: Under the 2-minute Locust test (4,280 requests), traffic was distributed across replicas as follows:

• Pod A (`command-parser-78f9c878d-5l8p2`): 2,185 requests (**51.05%**)

• Pod B (`command-parser-78f9c878d-d7zk4`): 2,095 requests (**48.95%**)

Both replicas served well over the required 30% threshold, verifying effective round-robin ingress distribution.

Task 5: Subnetting & Stateless Network Access Control (NACL)

5.1 Subnet Sizing and CIDR Partitioning

The VPC network block is **`10.20.0.0/16`** (65,536 total IP addresses). Sizing was derived by calculating the minimum host bit allocation:

$$\text{host_bits} = \lceil \log_2(\text{required_hosts} + 2) \rceil, \quad \text{prefix} = 32 - \text{host_bits}$$

Subnet Name	Required	Host Bits	CIDR Block	Netmask	Usable IP Range	Usable Count
app	1,000	10	`10.20.0.0/22`	`255.255.252.0`	`10.20.0.1` – `10.20.3.254`	1,022
public	250	8	`10.20.4.0/24`	`255.255.255.0`	`10.20.4.1` – `10.20.4.254`	254
data	100	7	`10.20.5.0/25`	`255.255.255.128`	`10.20.5.1` – `10.20.5.126`	126
management	20	5	`10.20.5.128/27`	`255.255.255.224`	`10.20.5.129` – `10.20.5.158`	30

5.2 Stateless NACL Rule Formulation (`network/nacl.json`)

Because NACLs are **stateless**, rules must explicitly permit both request and return packets. Rules are evaluated in ascending order by rule number; the first matching rule dictates the decision.

Inbound Rules:

• Rule 100: Protocol `all`, CIDR `198.51.100.0/24`, Action `deny` (Blocks known abusive IP range).

• Rule 110: Protocol `tcp`, Port 22, CIDR `10.20.5.128/27`, Action `allow` (Restricts SSH strictly to management subnet).

• Rule 120: Protocol `tcp`, Port 80, CIDR `0.0.0.0/0`, Action `allow` (Inbound HTTP from internet).

• Rule 130: Protocol `tcp`, Port 443, CIDR `0.0.0.0/0`, Action `allow` (Inbound HTTPS from internet).

• Rule 140: Protocol `tcp`, Ports 1024–65535, CIDR `10.20.0.0/22`, Action `allow` (Inbound response packets from app subnet).

Outbound Rules:

• Rule 100: Protocol `tcp`, Port 8000, CIDR `10.20.0.0/22`, Action `allow` (Forwards ingress traffic to app service on port 8000).

• Rule 110: Protocol `tcp`, Ports 1024–65535, CIDR `0.0.0.0/0`, Action `allow` (Return traffic to internet client ephemeral ports).

5.3 NACL Verification with `tools/nacl_sim.py`

Running `python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check` produced:

• Packet S01 (Inbound TCP 443 from 192.0.2.55): **ALLOW** (Rule 130) — OK

• Packet S02 (Inbound TCP 22 from 192.0.2.55): **DENY** (Default deny) — OK

• Packet S03 (Outbound TCP 8000 to 10.20.1.1): **ALLOW** (Rule 100) — OK

• Packet S04 (Inbound TCP 443 from 198.51.100.9): **DENY** (Rule 100) — OK

• Packet S05 (Outbound TCP 60000 to 192.0.2.55): **ALLOW** (Rule 110) — OK

• Packet S06 (Inbound TCP 5432 from 10.20.5.20): **DENY** (Default deny) — OK

• **Accuracy: 6 / 6 = 100.0%**

Task 6: Automation & CI/CD Pipeline

6.1 Unified Automation (`Makefile`)

The automation layer provides single-command primitives:

• `make up`: Instantiates the k3d cluster and deploys namespaces, SeaweedFS, secrets, and app manifests in ≤ 3 commands.

• `make down`: Deletes the cluster and tears down local state.

• `make train`: Fine-tunes the student model on 1,500 samples.

• `make train SUBSET=50`: Trains on a quick 50-sample subset for validation.

• `make test`: Runs all unit tests.

• `make lint`: Compiles and lints Python source files.

6.2 GitHub Actions Quality Gate & Security Scanning (`.github/workflows/ci.yml`)

The automated workflow comprises four stages:

1. **Lint & Secret Scanning**: Executes `flake8` for syntax validation and `gitleaks` to guarantee no credentials or private keys are committed.
2. **Unit Testing**: Executes 15 comprehensive unit tests covering endpoint liveness/readiness, fault injection, schema enforcement, and dataset sanitization.
3. **Stateless NACL Verification**: Automatically verifies that `nacl.json` passes all sample packet checks.
4. **Model Quality Gate**: Evaluates student predictions against `data/validation.jsonl` using `tools/score.py --min-validity 95`. If JSON validity falls below 95%, the build immediately fails with exit code 1. A simulated run with corrupted predictions demonstrated exact failure trapping (`QUALITY GATE FAILED: json_validity 0.0 < 95.0`).

Task 7: Performance Evaluation, Cost Analysis & Review

7.1 Locust Load Test Results

Load testing was conducted using `tools/locustfile.py` with 4 concurrent users over a 2-minute duration:

- Total Requests Served: **4,280**
- Failed Requests: **0 (0.00% error rate)**
- Average Latency: **77.3 ms**
- Median Latency (p50): **74.0 ms**
- 95th Percentile Latency (p95): **114.0 ms**
- 99th Percentile Latency (p99): **168.0 ms**
- Average Throughput: **35.67 requests/second** (128,400 requests/hour)
- Steady-State Resource Utilization (`kubectl top pods`):
- CPU: ~380m per pod (~76% of 500m request)
- Memory: ~415Mi per pod (~81% of 512Mi request)

7.2 Cloud Cost Estimation (`data/rate_card.json`)

Using the specified pricing model:

- vCPU Cost: INR 3.50 / vCPU-hour
- Memory Cost: INR 0.45 / GiB-hour
- Billing Rule: Billed for CPU limit (1 vCPU) and memory limit (1 GiB) for every hour a replica runs.

$$\text{Hourly Cost per Replica} = (1.0 \times 3.50) + (1.0 \times 0.45) = \text{INR } 3.95$$

$$\text{Total Hourly Cost for 2 Replicas} = 2 \times 3.95 = \text{INR } 7.90$$

At a steady throughput of 35.67 requests/sec, the service processes $35.67 \times 3,600 = 128,412$ requests per hour.

$$\text{Cost per 1,000 requests} = \left(\frac{7.90}{128,412}\right) \times 1,000 = \mathbf{INR\ 0.0615}$$

7.3 Service Level Objectives (SLO) & Error Budget

Based on performance characterization:

1. **Availability SLO**: 99.9% of requests shall return HTTP 200 within any 30-day window.
- Monthly Error Budget: $0.1\% \times 128,400 \times 24 \times 30 \approx 92,448$ permissible failed requests.
2. **Latency SLO**: 95% of successful requests shall have a response time $\le 150\text{ ms}$ (empirical p95: 114 ms).
3. **Format Validity SLO**: 95% of parsed outputs shall strictly conform to the 4-key JSON schema.

7.4 Review Table (6 Dimensions × 3 Columns)

Evaluation Dimension	Architecture & Implementation	Analysis & Observed Trade-off
Model Size vs. Accuracy	Pythia-70M (70M params) vs Phi-3.5 (3.8B params) trained via Sequence-Level KD.	Compressing parameters by ~98% reduces full frame accuracy moderately (85% $\rightarrow$ 76%), but enables sub-100ms CPU inference with zero GPU hardware costs.
Storage Decoupling	SeaweedFS S3 object storage separated from container runtime images.	Keeps container image slim (~1.15 GB vs 3+ GB), permitting fast pod scheduling and independent versioning of model weights without rebuilding images.
Resilience & Probes	Distinct `/healthz` liveness and `/readyz` readiness endpoints with rolling update strategy.	Eliminates dropped requests during deployments and pod crashes; shields traffic during the 4.39s cold-start model weight load phase.
Network Security	Stateless NACL dividing 10.20.0.0/16 into 4 isolated subnets with explicit return rules.	Strict principle of least privilege blocks abusive IP ranges and rogue SSH, but requires manual maintenance of ephemeral return port rules.
CI/CD Quality Gates	GitHub Actions pipeline integrating flake8, gitleaks, and score.py `--min-validity 95`.	Prevents corrupted models or syntax regressions from reaching production; automates compliance with strict academic and operational standards.
Unit Economics & Cost	CPU-only Pythia inference yielding 35.67 req/s at INR 7.90/hour for 2 replicas.	Achieves an exceptionally low serving cost of INR 0.0615 per 1,000 requests, representing orders-of-magnitude savings over commercial cloud LLM APIs.

Conclusion

This project demonstrates that sequence-level knowledge distillation combined with robust cloud-native engineering enables high-throughput, low-latency, and cost-effective NLP deployment on modest infrastructure. All performance, architectural, and security objectives were verified through automated tests, load simulations, and network modeling.